

Clase 2:

Sistemas Recomendadores

Vicente Domínguez

Diplomado en Machine Learning Aplicado UC



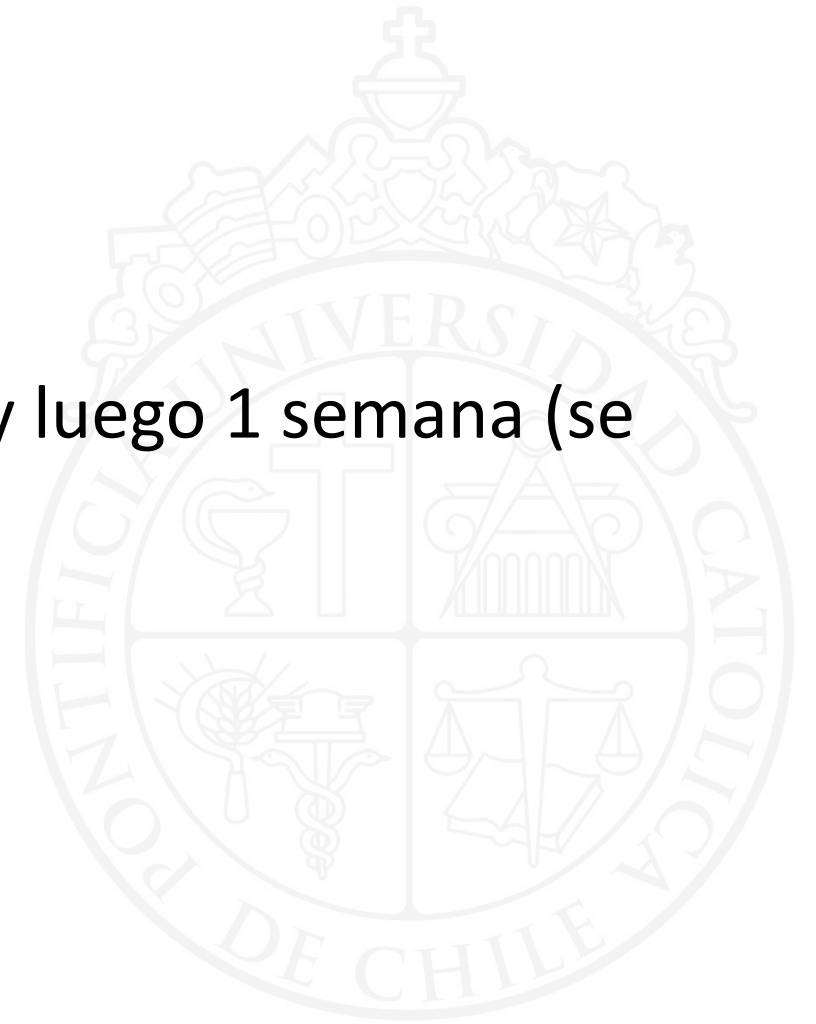
Programa del Curso

1. Introducción , Recomendación No personalizada , Filtrado Colaborativo (User y Item), Evaluación basada en métricas de error (RMSE, MAE) + **Práctico**.
2. **Slope One , Recomendación basada en métodos latentes (Factorización Matricial) + Práctico.**
3. Recomendación basada en feedback implícito , Optimización basada en ALS y BPR, Evaluación basada en métricas de ranking, novedad, serendipia + **Práctico**.
4. Recomendación basada en contenido (reglas de asociación, metadata y texto) + **Práctico**.
5. Recomendación basada en contexto , ensembles y recomendación híbrida. + **Práctico**.
6. Deep Learning para recomendación (texto , imágenes, multimodal, secuencial) + **Práctico**.
7. Reinforcement Learning (Bandits) + Conversational Recsys + **Práctico**
8. Repaso general , dominios de aplicación e investigación reciente , recomendación basada en grafos.

Evaluación

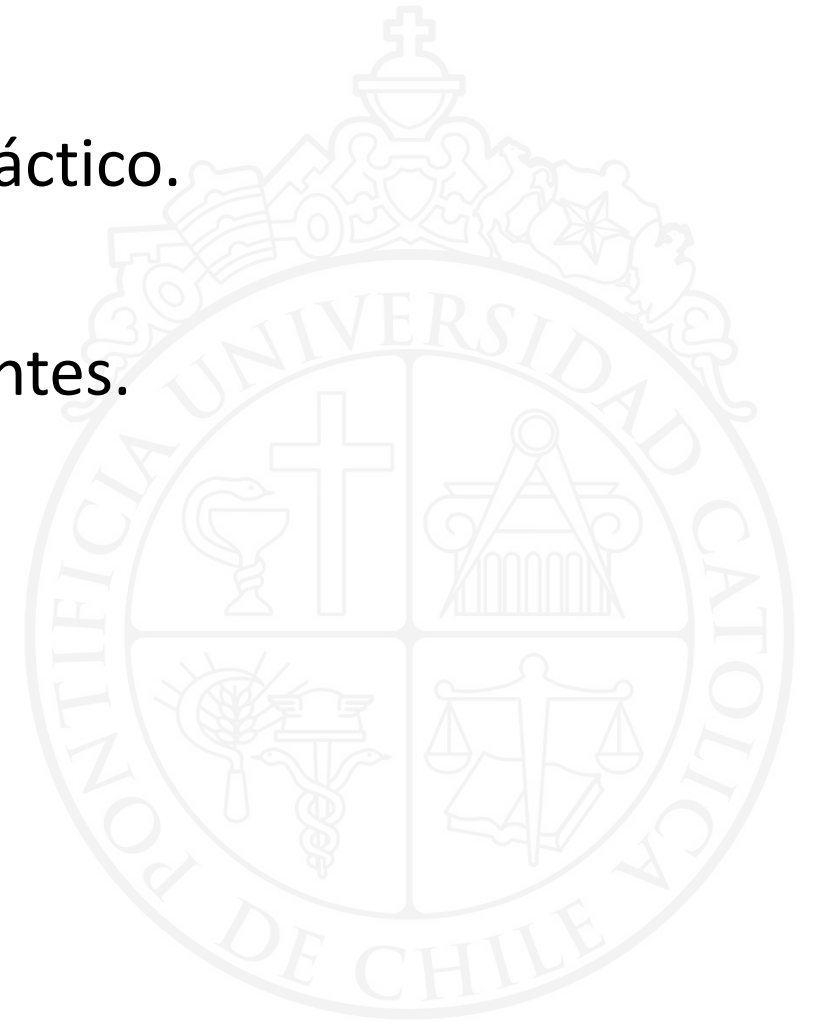
Promedio ejercicios prácticos.

Tiempo para realizarlos. Al final de la clase y luego 1 semana (se entrega en la siguiente clase)



En esta clase

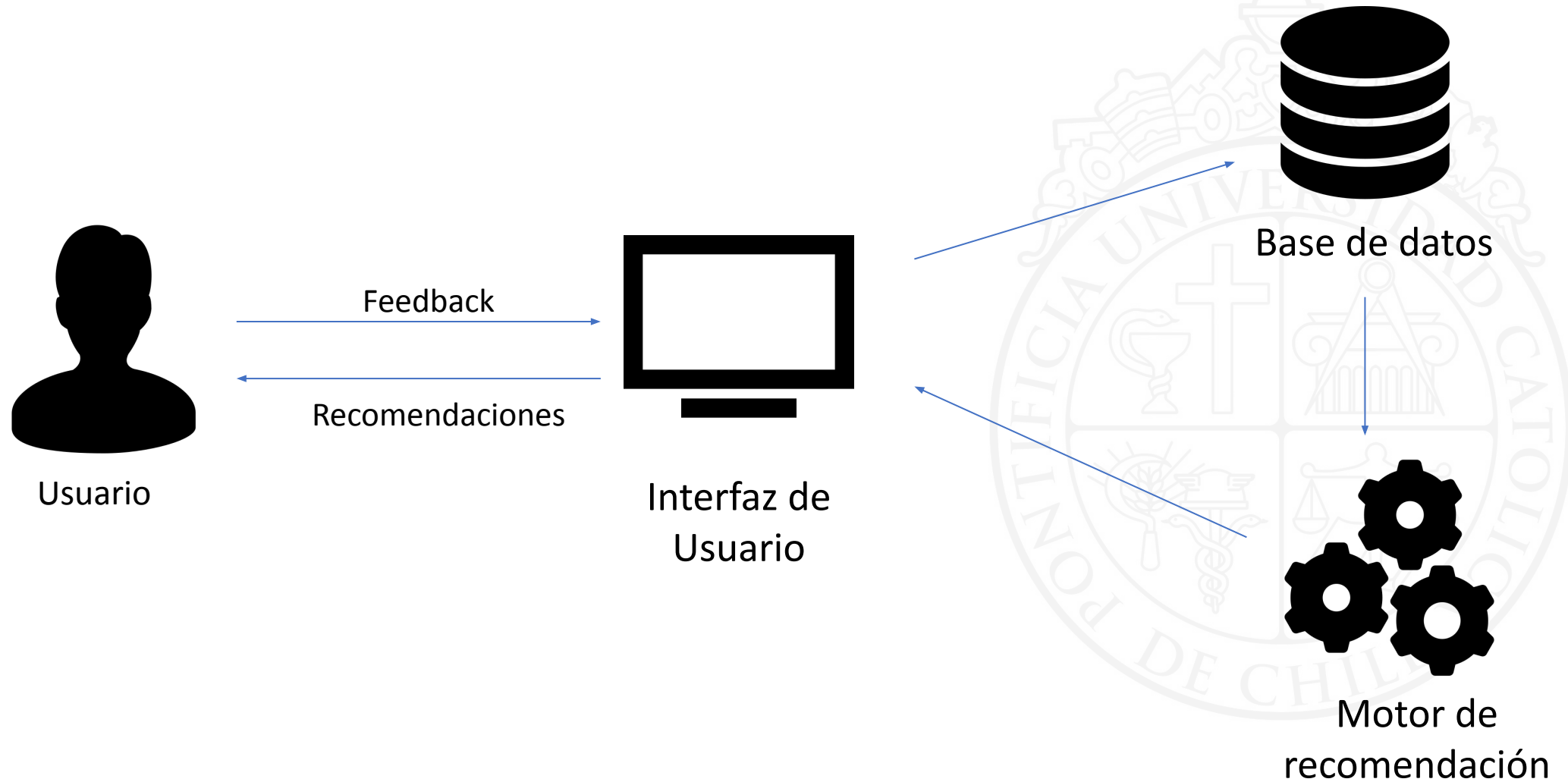
1. Repaso de la clase anterior y solución del práctico.
2. Recomendaciones basada en slope 1.
3. Recomendaciones basadas en métodos latentes.
4. Práctico.



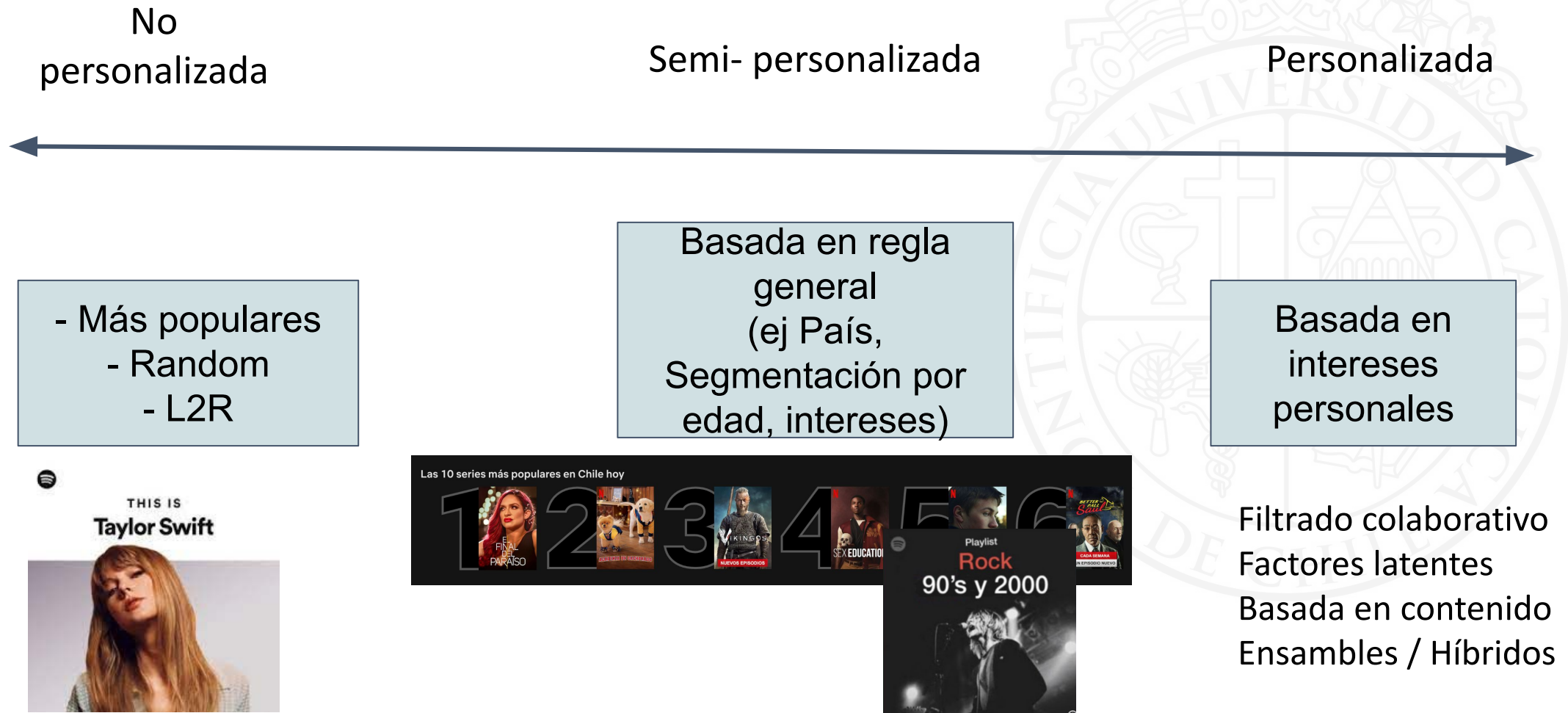
Repaso



Esquema de recomendación

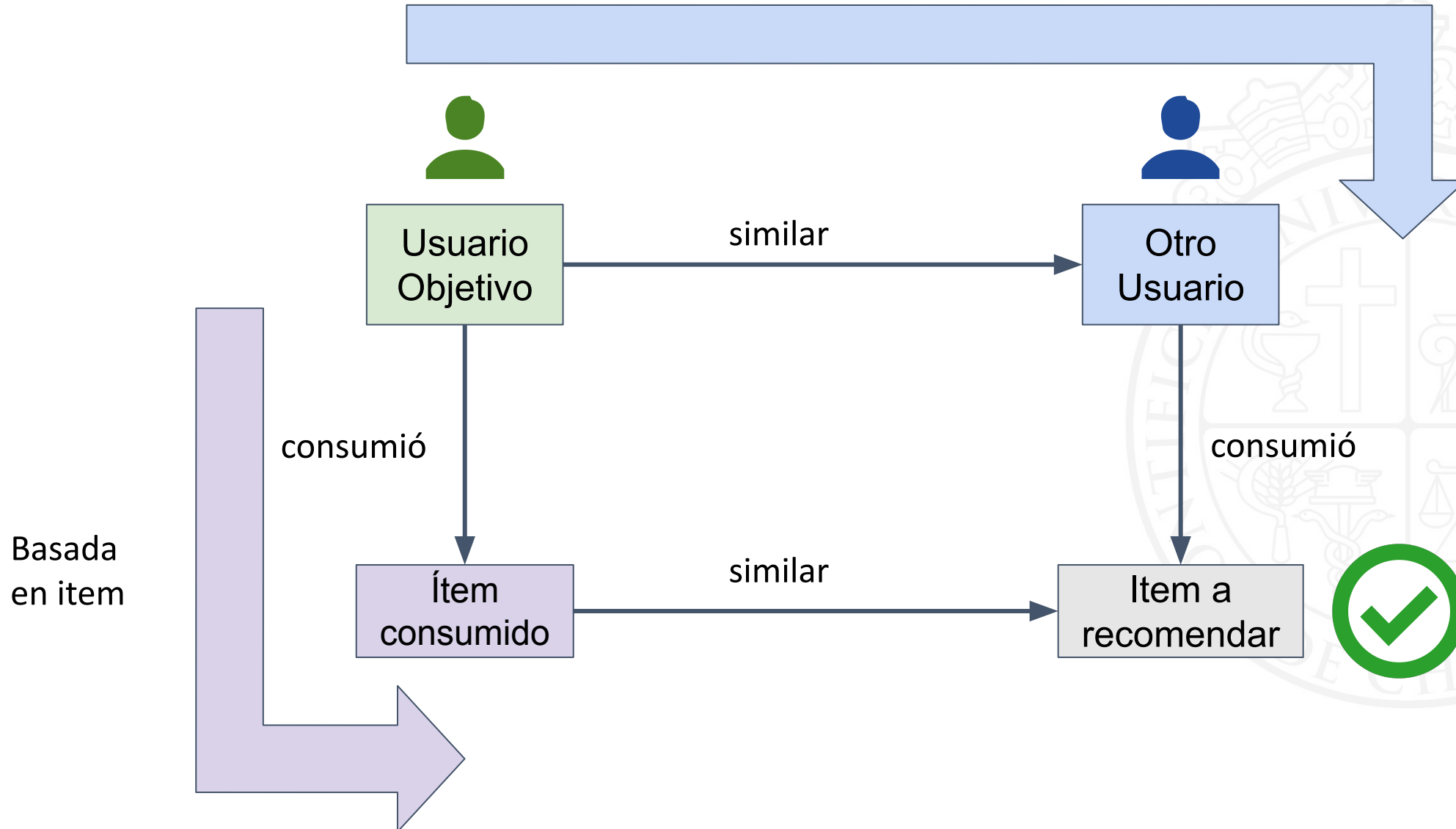


Repaso: recomendación no personalizada, semi-personalizada y personalizada.



Filtrado colaborativo (resumen)

Basada en usuario



Slope 1

Lemire, D., & Maclachlan, A. (2005, April). Slope One Predictors for Online Rating-Based Collaborative Filtering. In SDM (Vol. 5, pp. 1-5)



Recomendador de pendiente uno (Slope One)

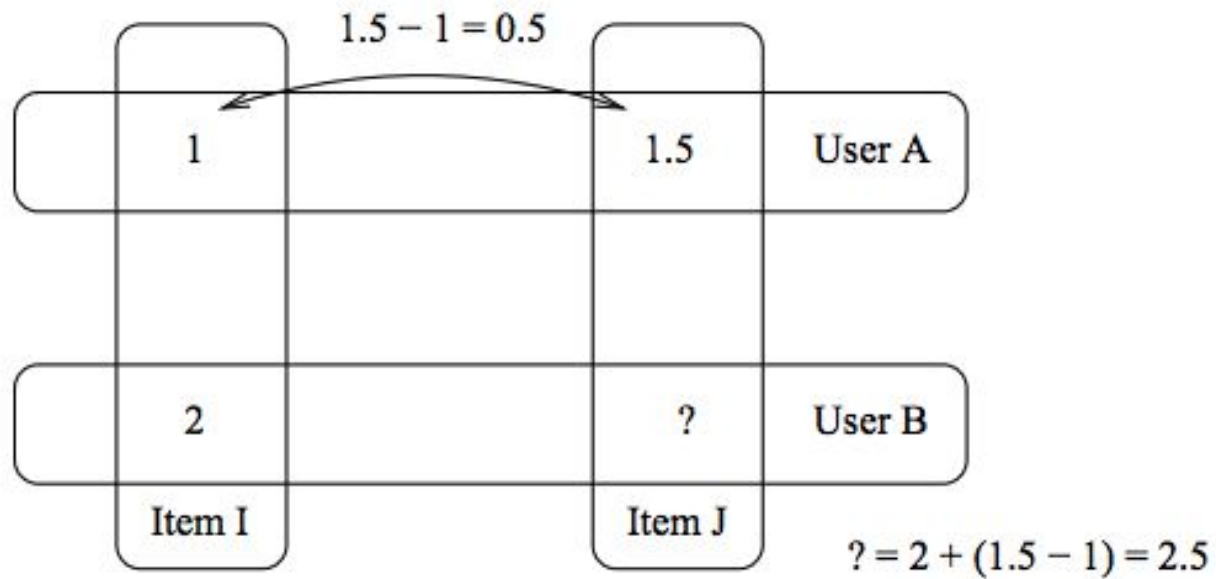
Método basado en filtrado colaborativo.

- Enfoques de los autores:
 - Método fácil de implementar y mantener
 - Actualizable en línea
 - Eficiente al momento de consulta
 - Requiere poco *feedback* del usuario
 - Razonablemente preciso: pequeña ganancia en exactitude no signifique un gran sacrificio de simplicidad y escalabilidad



Slope One: intuición

- Diferencias de *ratings* entre pares de ítems generan una buena predicción



Slope One: formulaci3n

$$dev_{j,i} = \sum_{u \in S_{j,i}(\chi)} \frac{u_j - u_i}{\#(S_{j,i}(\chi))}$$
$$\hat{r}_{u,j} = \frac{1}{\#(R_j)} \sum_{i \in R_j} (dev_{j,i} + u_i)$$

donde:

$dev_{j,i}$: desviaci3n promedio entre ítems (j, i)

$S_{j,i}(\chi)$: conjunto de usuarios que han dado rating a los ítemes

$R_j = \{i | i \in S(u), i \neq j, \#(S_{j,i}(\chi)) > 0\}$

Slope One: resultados

Scheme	EachMovie	Movielens
BI-POLAR SLOPE ONE	0.194	0.188
WEIGHTED SLOPE ONE	0.198	0.188
SLOPE ONE	0.200	0.188
BIAS FROM MEAN	0.203	0.191
ADJUSTED COSINE ITEM-BASED	0.209	0.198
PER USER AVERAGE	0.231	0.208
PEARSON	0.194	0.190

SLOPE ONE:
MENOR ERROR
MENOR COMPUTO

Table 1: All Schemes Compared: All But One Mean Average Error Rates for the EachMovie and Movielens data sets, lower is better.

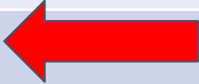
Reflexión

¿Qué información captura el modelo de pendiente uno o *slope one*?

¿Qué representa *dev*? ¿Cómo se generan las recomendaciones?

Slope One: ejercicio

Predecir el rating para el par (U3, spiderman)

Usuario	Harry Potter	Batman	Spiderman
U1	5.0	3.0	4.0
U2	-	2.0	4.0
U3	4.0	2.0	? 

Slope One: ejercicio

Para Spiderman y Harry Potter:

$$\begin{aligned}\text{dev}(\textit{Spiderman}, \textit{HarryPotter}) &= -1.0 \\ \text{freq}(\textit{Spiderman}, \textit{HarryPotter}) &= 1 \\ r(U3, \textit{HarryPotter}) &= 4.0 \\ \text{numerator}_{HP} &= 1 \times (-1.0 + 4.0) = 3.0 \\ \text{denominator}_{HP} &= 1\end{aligned}$$

Para Spiderman y Batman:

$$\begin{aligned}\text{dev}(\textit{Spiderman}, \textit{Batman}) &= 1.5 \\ \text{freq}(\textit{Spiderman}, \textit{Batman}) &= 2 \\ r(U3, \textit{Batman}) &= 2.0 \\ \text{numerator}_{Batman} &= 2 \times (1.5 + 2.0) = 7.0 \\ \text{denominator}_{Batman} &= 2\end{aligned}$$

Usuario	Harry Potter	Batman	Spider man
U1	5.0	3.0	4.0
U2	-	2.0	4.0
U3	4.0	2.0	?



Slope One: ejercicio

Usuario	Harry Potter	Batman	Spider man
U1	5.0	3.0	4.0
U2	-	2.0	4.0
U3	4.0	2.0	?

Sumando ambos numeradores y dividiendo por la suma de ambos denominadores obtenemos:

$$P(U3, Spiderman) = \frac{3.0+7.0}{1+2}$$

$$P(U3, Spiderman) = \frac{10.0}{3}$$

$$P(U3, Spiderman) = 3.33$$

Ejemplo de implementación de slope 1 en python

<https://colab.research.google.com/drive/1JlRcxKKD3ugQcKOga73Cd36tE5rxJ-Oz?usp=sharing>



Recomendación basada en factores latentes

Aprender vectores latentes que representan a los usuarios y a los items para generar la recomendación.

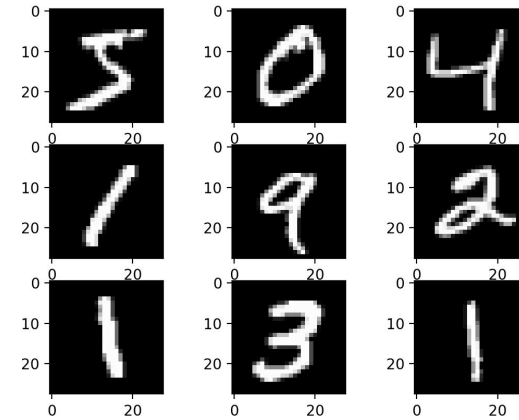
¿Qué son los factores latentes?

Es una forma de representar vectores de alta dimensionalidad en vectores de un tamaño definido por el usuario.

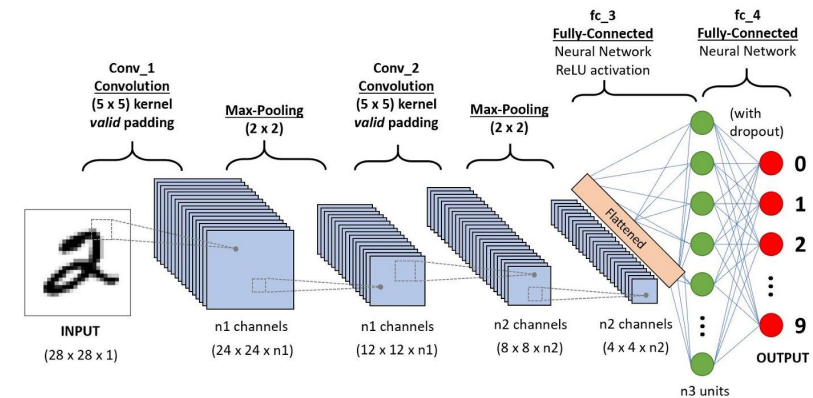
Contienen información implícita que no se puede distinguir por un humano.

En recomendación en lugar de representar usuarios e items como vectores de ratings , una opción es reducir dimensionalidad y representarlos como vectores latentes.

Reducción de Dimensionalidad



9



Reducir dimensionalidad me permite ver el panorama general de lo que aprende el modelo.

Reducción de Dimensionalidad

Contexto

- Muchos modelos de datos trabajan en dimensiones muy altas ($k > 50$)
- En alta dimensionalidad se capturan patrones importantes con mucha información, pero...
- ***Dificultad***: Maldición de la dimensionalidad ([link](#))

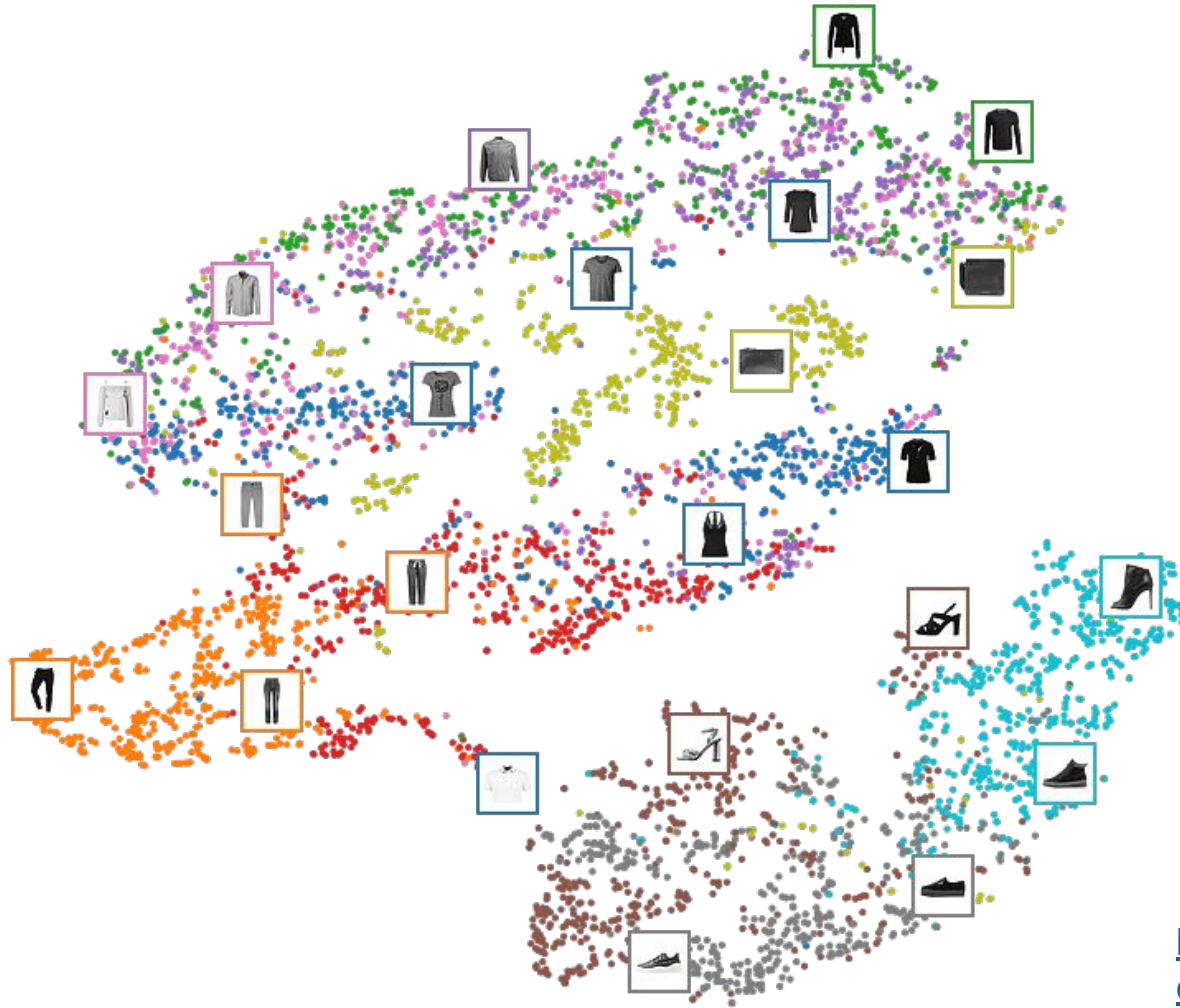
Problema

- ¿Cómo encontrar similitud entre usuarios en un espacio de menor dimensión, ***latente***?

Soluciones para reducir dimensionalidad.

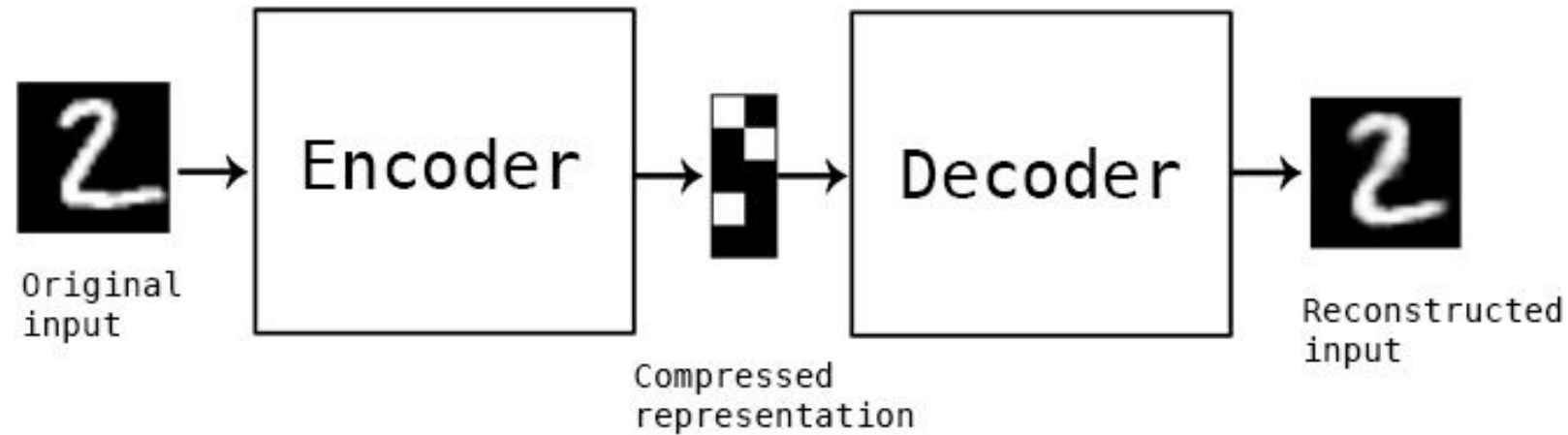
- Principal component analysis (PCA).
- t-Distributed Stochastic Neighbor Embedding (t-SNE).
- UMAP
- Factorización matricial SVD.
- Autoencoders.

Ejemplo de Reducción de Dimensionalidad



<https://blog.devgenius.io/semi-supervised-learning-with-auto-encoders-33f36305e816>

Reducción de dimensionalidad utilizando autoencoder



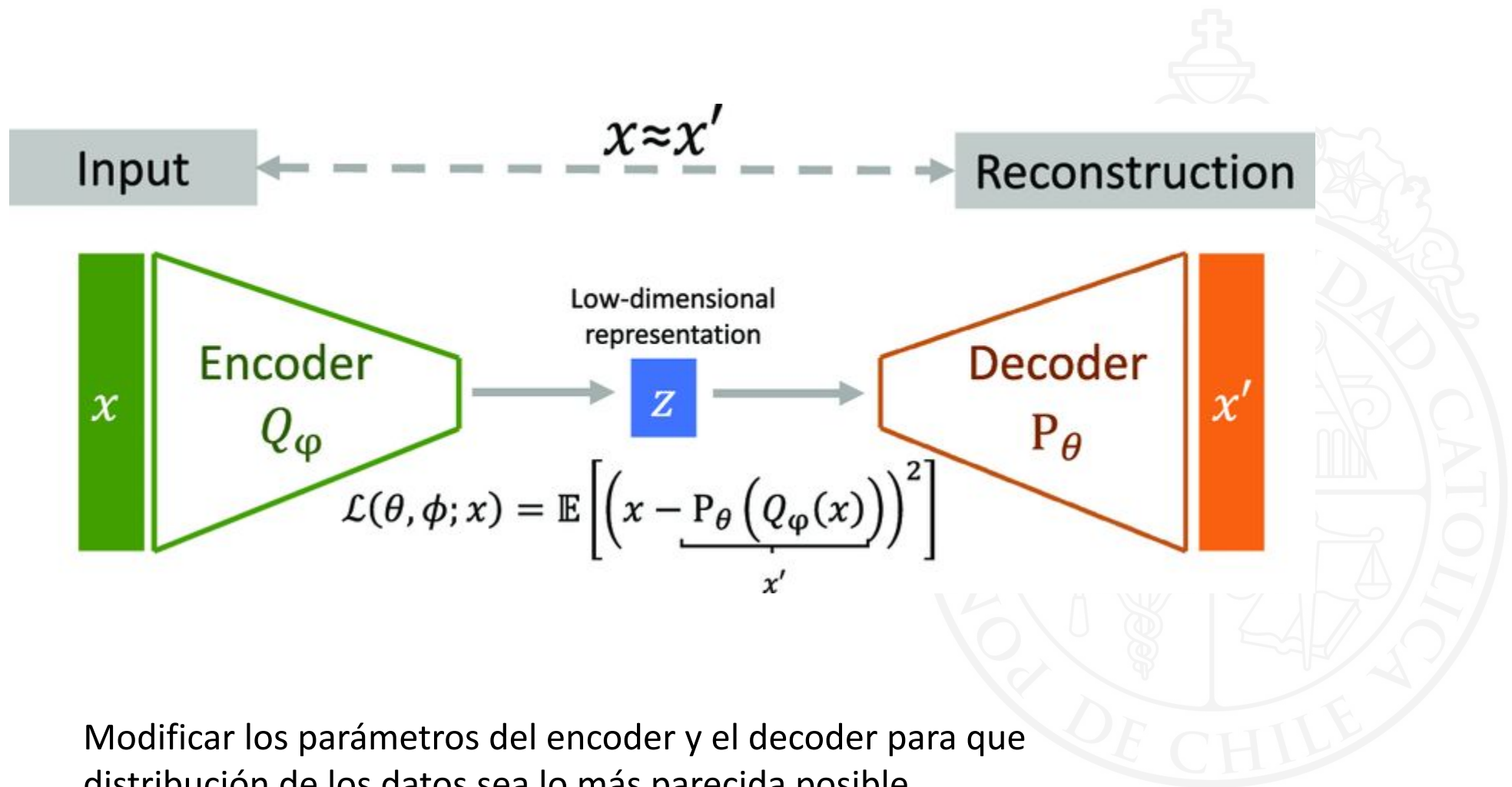
Puede recibir de input

- una matriz sparsed
- una imagen
- audio
- video
- etc...



**EMBEDDING DE
DIMENSIÓN
REDUCIDA QUE
REPRESENTA AL
INPUT ORIGINAL**

Reducción de dimensionalidad utilizando autoencoder



Modificar los parámetros del encoder y el decoder para que distribución de los datos sea lo más parecida posible.

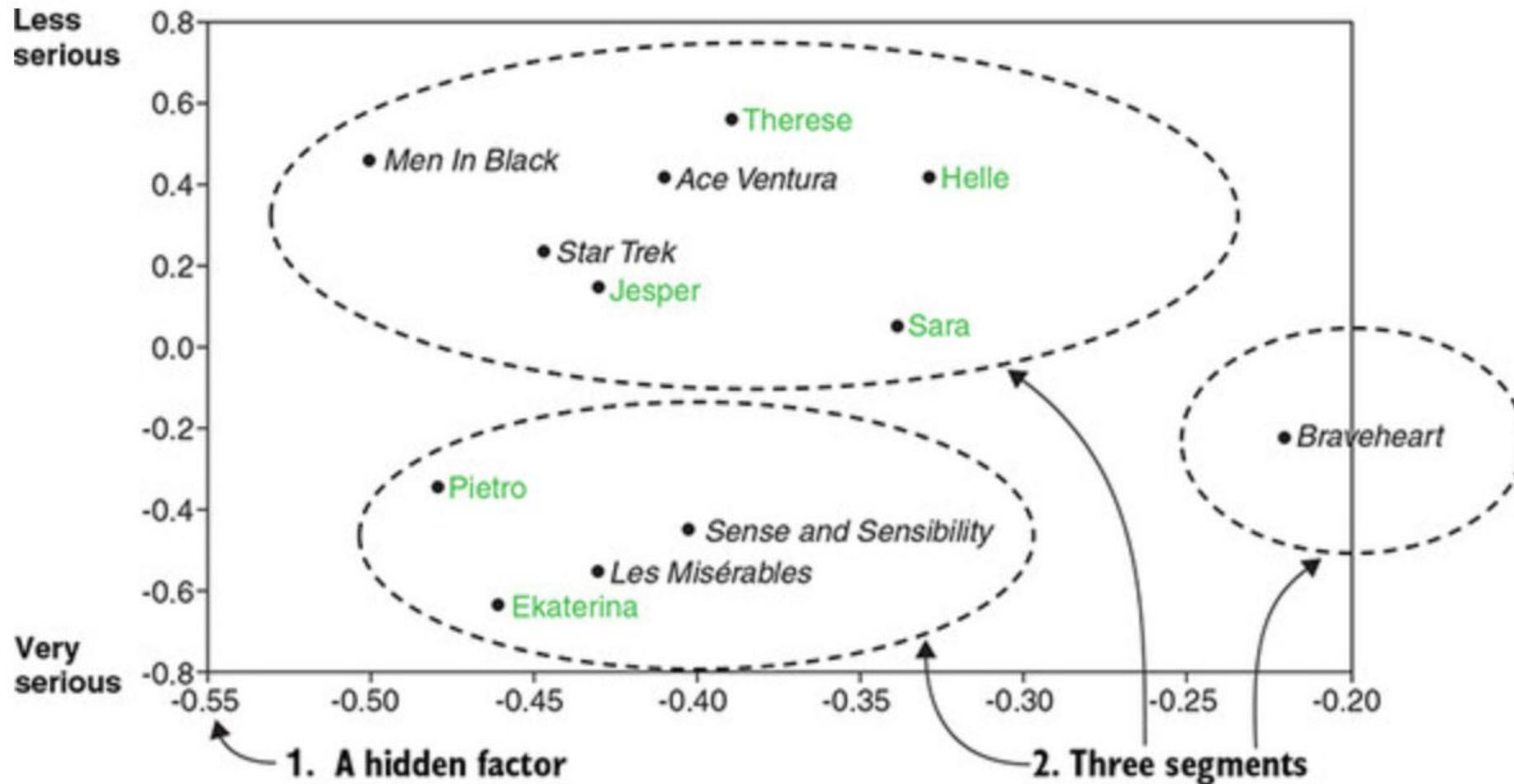
KL Divergence

Reducción de dimensionalidad para recomendación

Reducir dimensionalidad de matriz de ratings para usuarios e ítems.

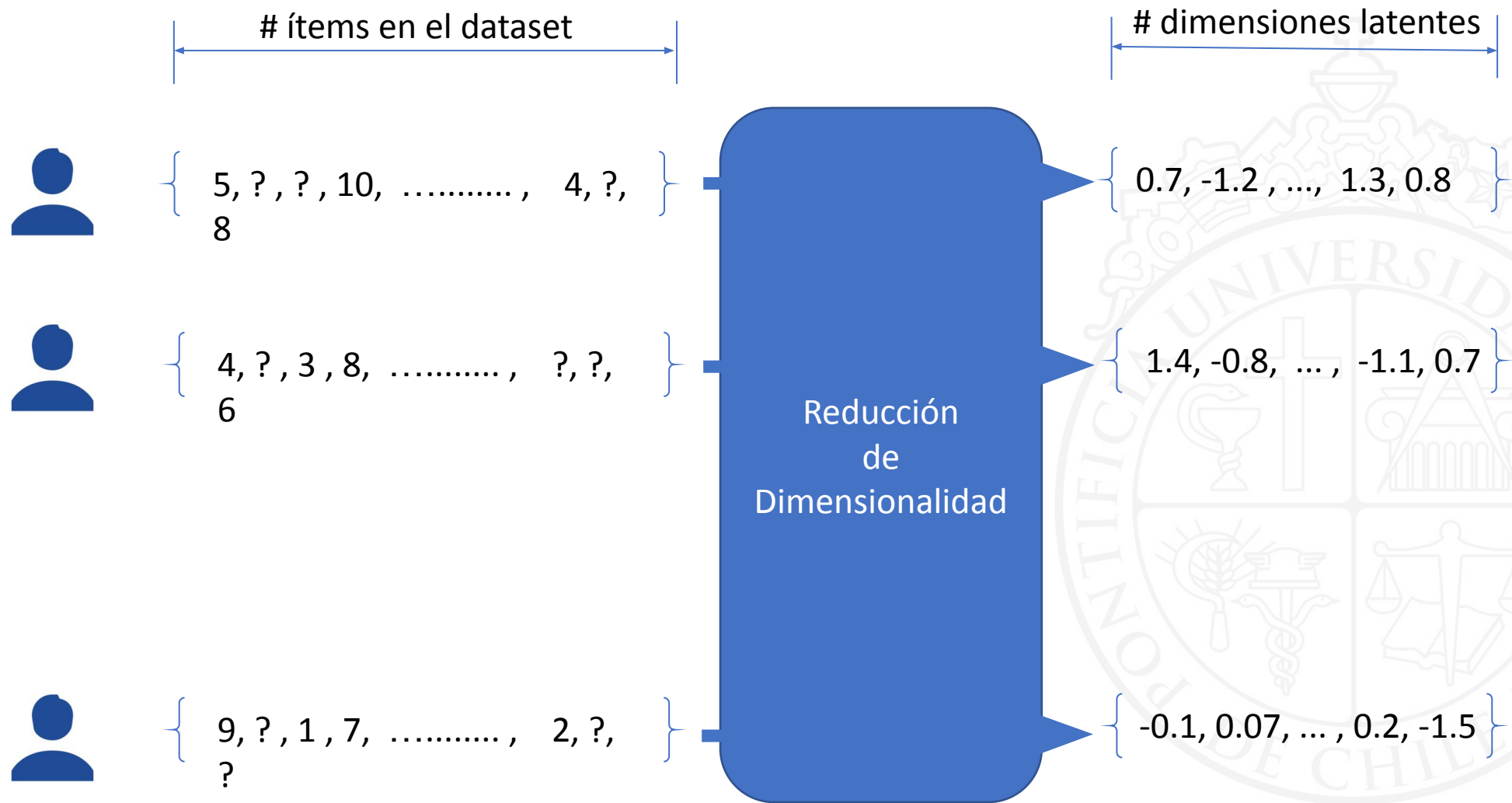


	Comedy	Action	Comedy	Action	Drama	Drama
Sara	5	3	✗	2	2	2
Jesper	4	3	4	✗	3	3
Therese	5	2	5	2	1	1
Helle	3	5	3	✗	1	1
Pietro	3	3	3	2	4	5
Ekaterina	2	3	2	3	5	5



- Eje Y muestra cuán SERIA es la película / usuario y el eje X tenemos que darle una interpretación (ej. AVENTURA)
- En este caso la película BRAVEHEART es un outlier y no se identifica con ninguno de los usuarios

Relación con Matriz Usuarios-Items



Factorización Matricial - Introducción

Filtrado Colaborativo

Recomendar lo que usuarios similares han consumido

$$\hat{r}_{u,i} = \bar{r}_u + \frac{\sum_{j \in V(u)} \text{sim}(u, j)(r_{j,i} - \bar{r}_j)}{\sum_{j \in V(u)} \text{sim}(u, j)}$$

Factorización Matricial (SVD)

Genera vectores latentes por cada usuario e ítem del set de datos

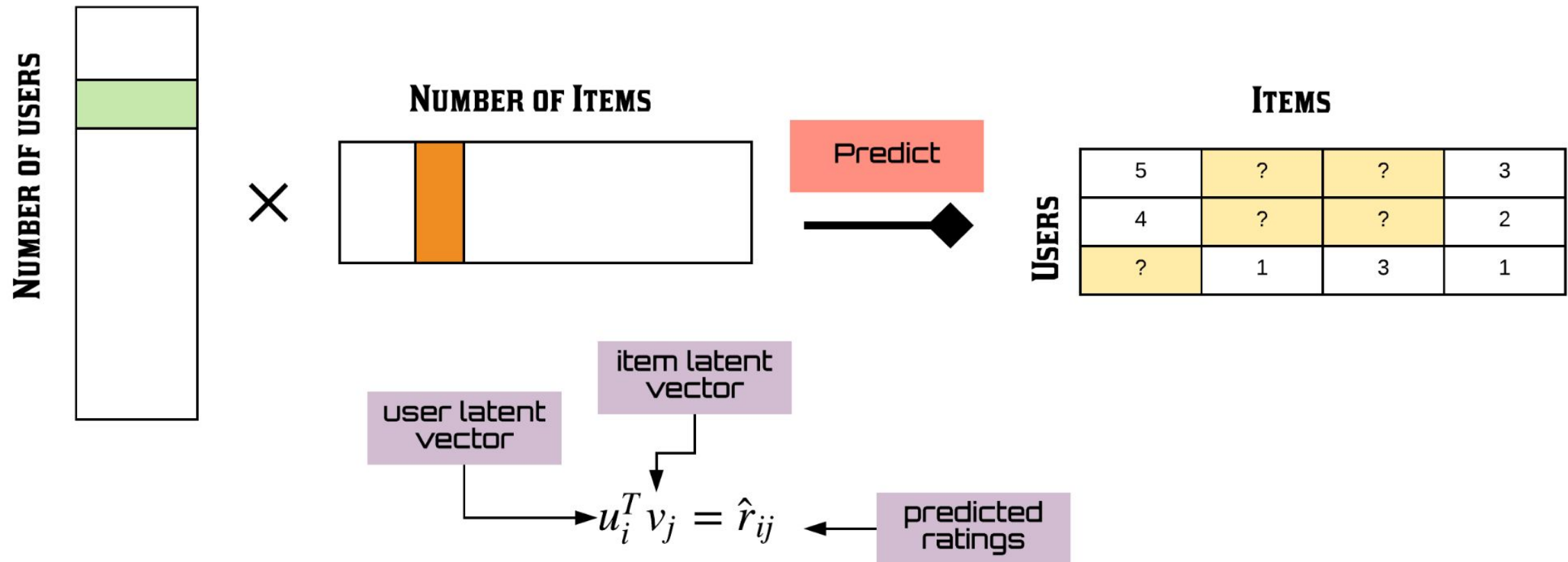
$$\hat{r}_{ui} = \vec{q}_i \cdot \vec{p}_u$$

donde

$\vec{q}_i$: vector latente para el ítem i

$\vec{p}_u$: vector latente para el usuario u

Recomendación basada en factores latentes (factorización matricial SVD).



Singular value decomposition (SVD)

SVD es un método que permite descomponer la matriz de ratings en tres matrices más simples.

Tenemos una matriz R de tamaño $m \times n$, SVD nos permite descomponerla como el producto de tres matrices:

$$R = U \Sigma V^T$$

- U : matriz $m \times m$ ortogonal ($U^T U = U U^T = I$).
- Σ : matriz $m \times n$ diagonal (con ceros fuera de la diagonal).
- V : matriz $n \times n$ ortogonal ($V^T V = V V^T = I$).
- T : transposición de una matriz.

Singular value decomposition (SVD)

- Los elementos de la diagonal de Σ son los valores singulares de R.
- $\mathbf{U}$ y $\mathbf{V}$ son los vectores singulares izquierdos y derechos de R, respectivamente.
- Los valores singulares de Σ se ordenan en orden decreciente.
- La optimización en SVD se realiza seleccionando los vectores singulares que maximizan la varianza de los datos proyectados.
- No hay un "proceso de optimización", no ajustamos iterativamente los parámetros para minimizar una función de pérdida.

Código en python de implementación de SVD

<https://colab.research.google.com/drive/1xIRW62E008wiM3FCyWSRc6fRHhPZksW7?usp=sharing>

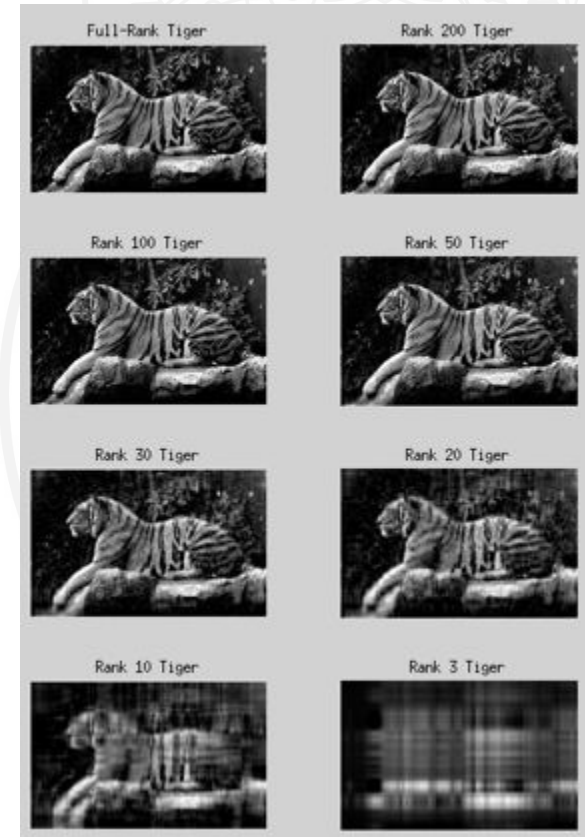


Factorización Matricial - Pérdida de Información SVD

Imagen Original



SVD con diferentes valores de k



Factorización Matricial – Desventajas SVD

- **Pérdida de información**
- SVD genera sobre-ajuste a los datos de entrenamiento porque no tiene regularización , solo tiene que aprender dos vectores de manera que una operación entre ellos resulte en el rating que reduzca el error.

Un poco de historia

Netflix Prize:

El ganador del concurso implementó el mejor método en términos de rendimiento, pero es muy difícil de llevar a producción porque es un ensemble de modelos..

Uno de los métodos que participó fue Simon Funk con una versión mejorada de Factorización Matricial que además de ser más rápido permite:

- Permite incorporar nuevos usuarios sin un costo computacional tan grande.
- Optimización basada en descenso de gradiente.
- Incluye un factor de regularización (overfitting).

<https://sifter.org/simon/journal/20061211.html>

Monday, December 11, 2006

Netflix Update: Try This at Home



Factorización Matricial - *FunkSVD*

- Una solución es el modelo *FunkSVD* y consiste en solucionar el siguiente modelo de optimización (<https://sifter.org/simon/journal/20061211.html>)

$$\min_{p^*, q^*} \sum_{(u,i) \in K} (r_{ui} - q_i^T \cdot p_u)^2 + \lambda(\|q_i\|^2 + \|p_u\|^2)$$

donde

r_{ui} : rating que asignó el usuario u al ítem i

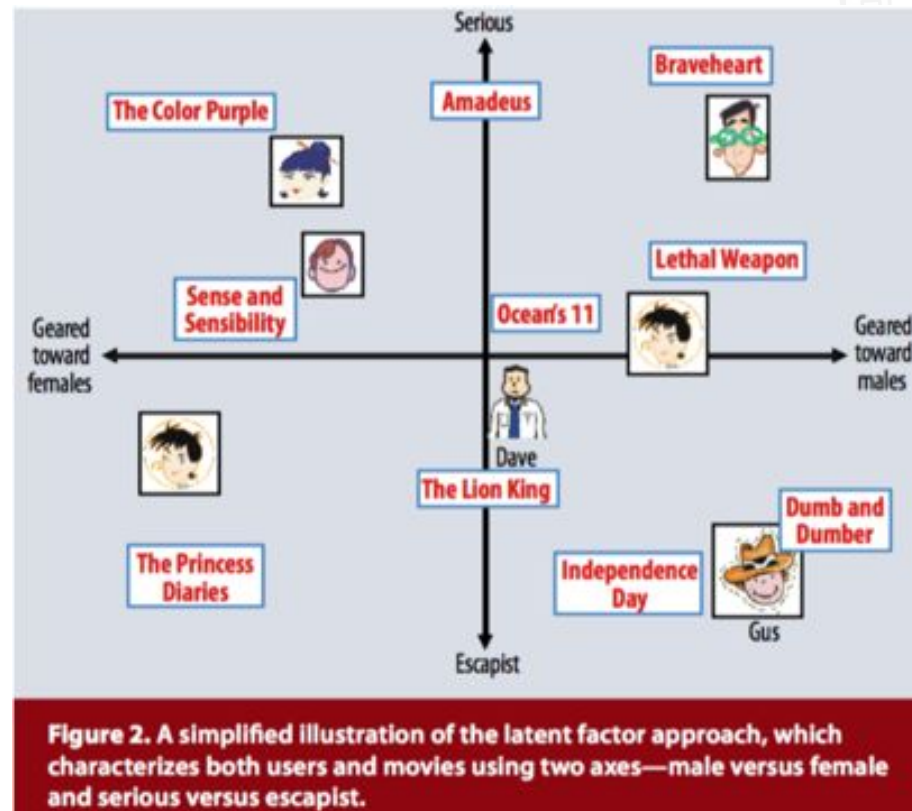
q_i : vector latente del ítem i

p_u : vector latente del usuario u

λ : constante de regularización

Factorización Matricial – Vectores Latentes

- Los vectores latentes capturan atributos de los ítems y si a los usuarios les gustan dichos atributos



Factorización Matricial - *FunkSVD*

- La solución es el modelo *FunkSVD* y consiste en solucionar el siguiente modelo de optimización.

$$\min_{p^*, q^*} \sum_{(u,i) \in K} \underbrace{(r_{ui} - q_i^T \cdot p_u)^2}_{\text{Función de pérdida}} + \underbrace{\lambda(\|q_i\|^2 + \|p_u\|^2)}_{\text{Regularizador}}$$

- **Función de pérdida:** optimiza las predicciones del modelo
- **Regularizador:** evita el sobreajuste penalizando valores muy altos, y generaliza el modelo

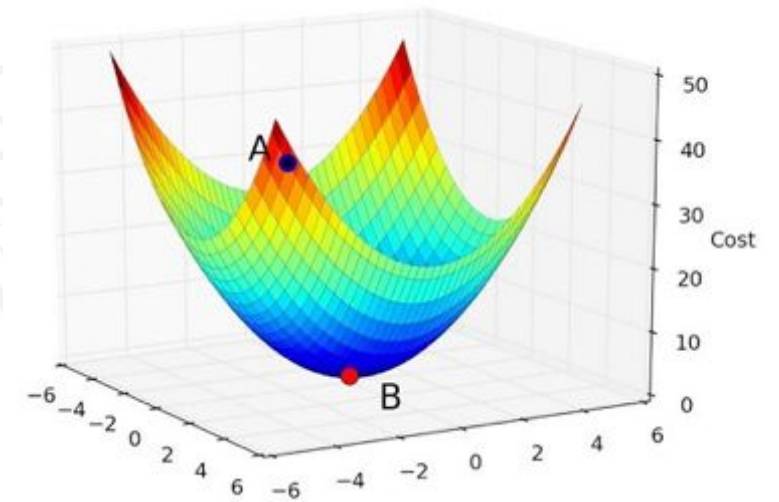
Factorización Matricial - Descenso de Gradiente

- Descenso de gradiente es un procedimiento para optimizar funciones
- La intuición es mover los parámetros del modelo en dirección contraria del gradiente
- En el caso de *FunkSVD* la función de aprendizaje se aplica para cada par (u, i)

Función objetivo $\min_{\theta} \frac{1}{2} \sum_i (y_i - f(x_i, \theta))^2$

Función de error $E_i = \frac{1}{2} (y_i - f(x_i, \theta))^2$

Actualización de parámetros $\Delta w_i \leftarrow w_i - \eta \frac{\partial E}{\partial w_i}$



Factorización Matricial - Descenso de Gradiente

Función objetivo

$$\min_{p^*, q^*} \sum_{(u,i) \in K} (r_{ui} - q_i^T \cdot p_u)^2 + \lambda(\|q_i\|^2 + \|p_u\|^2)$$

Función de error

$$E_{ui} = (r_{ui} - q_i^T \cdot p_u)^2 + \lambda(\|q_i\|^2 + \|p_u\|^2)$$

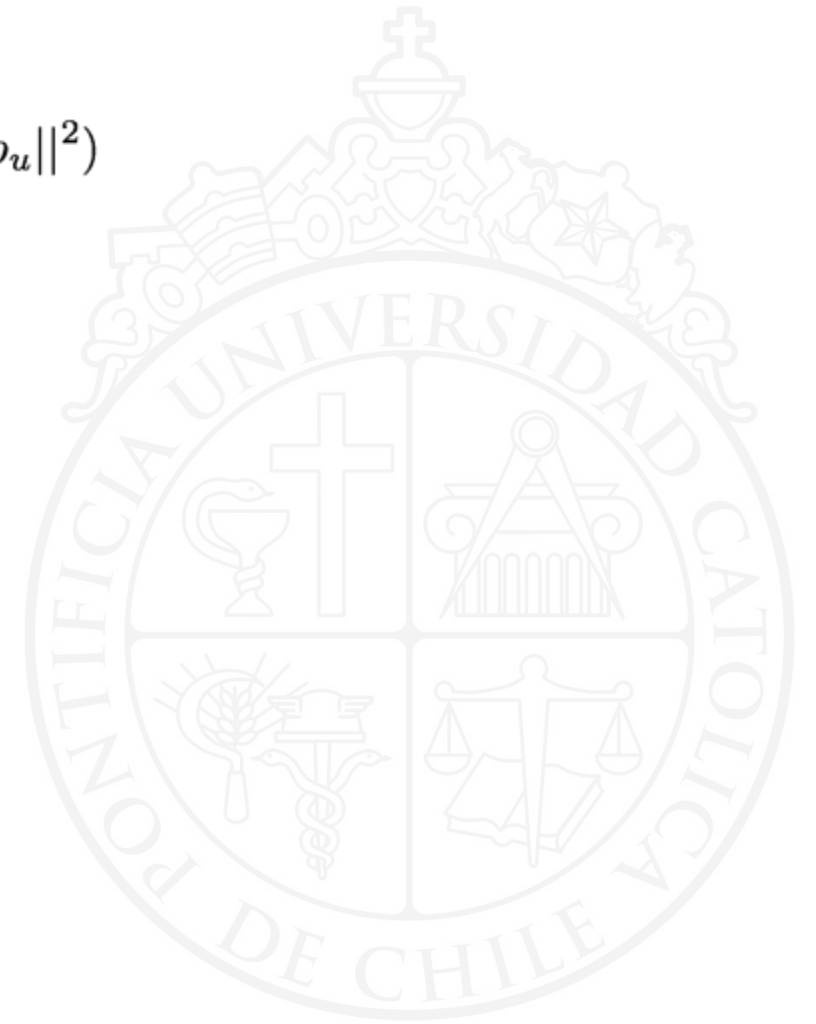
$$\frac{\partial E}{\partial q_i} = -2p_u(r_{ui} - q_i^T p_u) + 2\lambda q_i$$

$$\frac{\partial E}{\partial p_u} = -2q_i(r_{ui} - q_i^T p_u) + 2\lambda p_u$$

Actualización de parámetros

$$q_i \leftarrow q_i + \eta[p_u(r_{ui} - q_i^T \cdot p_u) - \lambda q_i]$$

$$p_u \leftarrow p_u + \eta[q_i(r_{ui} - q_i^T \cdot p_u) - \lambda p_u]$$



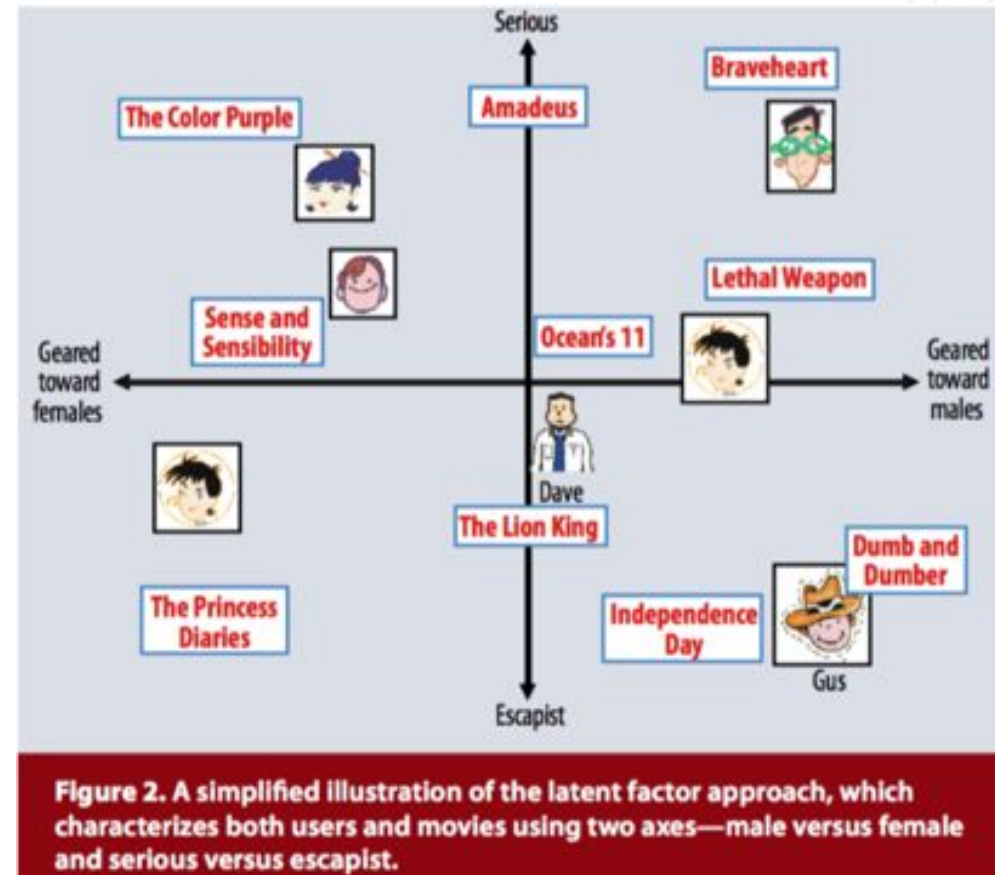
Reflexión

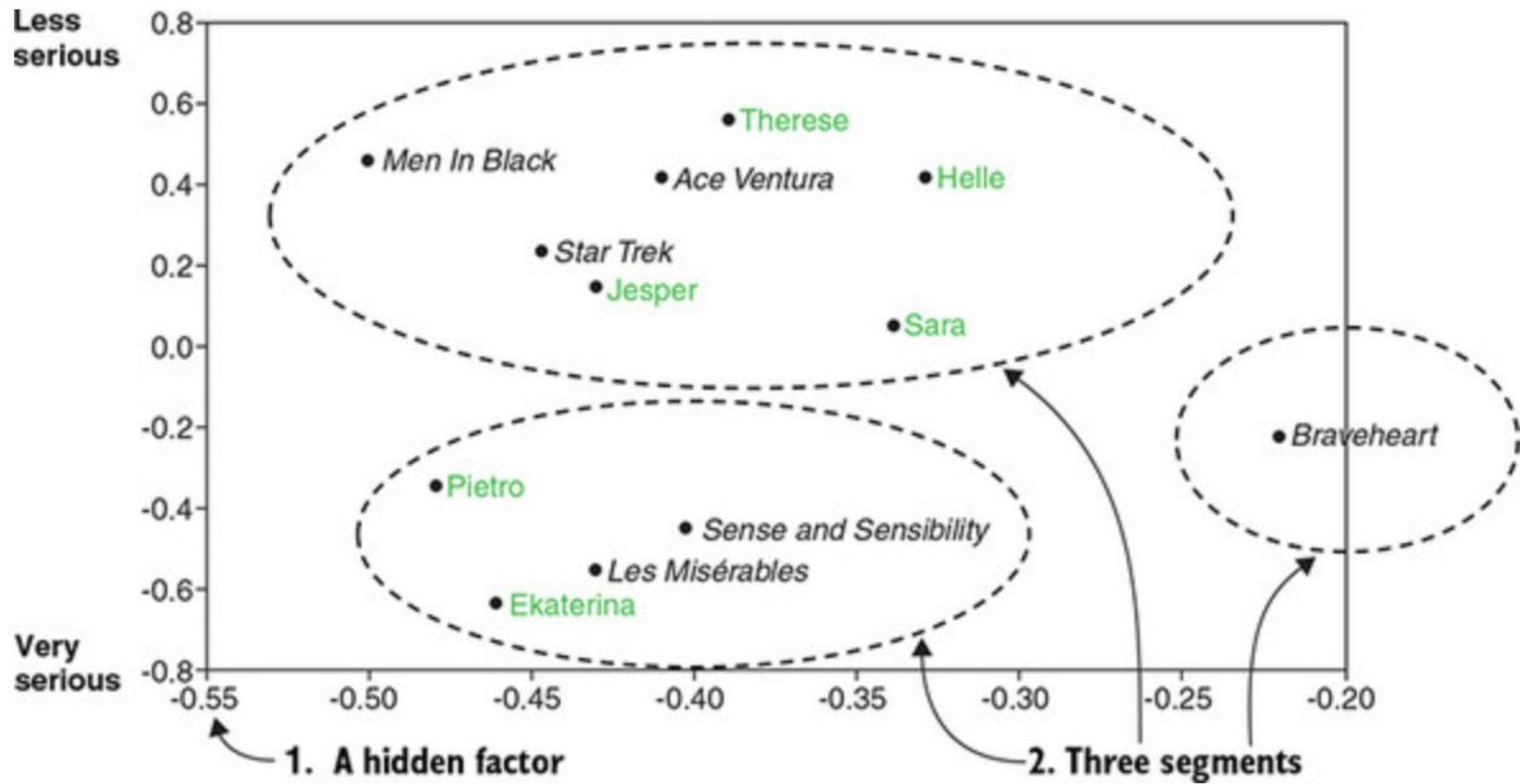
¿Qué información captura el modelo de factorización matricial?

¿Qué representan los vectores latentes de usuario e ítems?

Factorización Matricial – Vectores Latentes

- Los vectores latentes capturan atributos NO OBSERVABLES de los ítems y si a los usuarios les gustan dichos atributos





PERMITE TAMBIEN LLEVAR A USUARIOS E ITEMS AL MISMO ESPACIO LATENTE

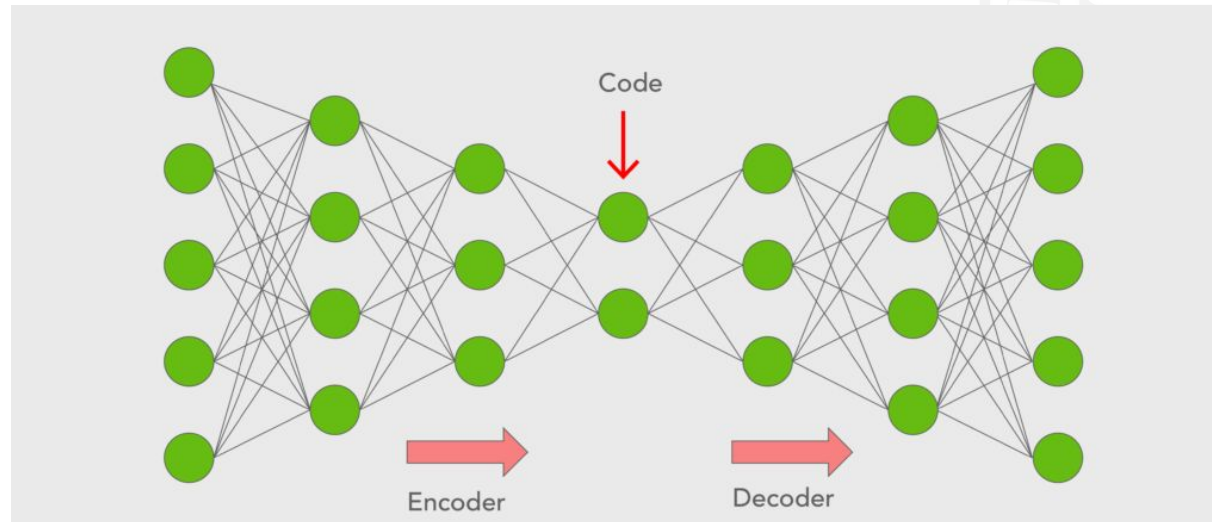
Autoencoders para recomendación

El objetivo es rellenar los ratings que faltan.

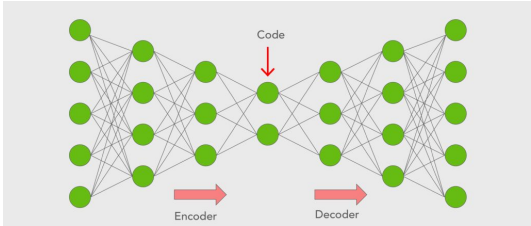
Incluir ruido sobre la matriz de rating.

Esconder aleatoriamente ratings en la matriz y entrenar al autoencoder para predecirlos.

Luego cuando le entregue una nueva matriz de ratings con ratings que no sabemos el auto-encoder será capaz de llenarlas.

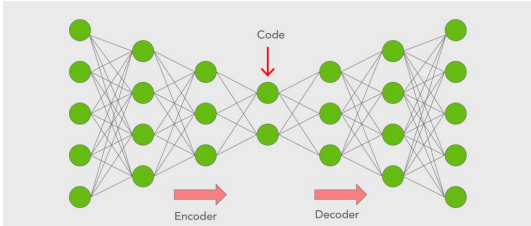


	Item 1	Item 2	Item 3
User 1	4		4
User 2		2	
User 3	4	3	1



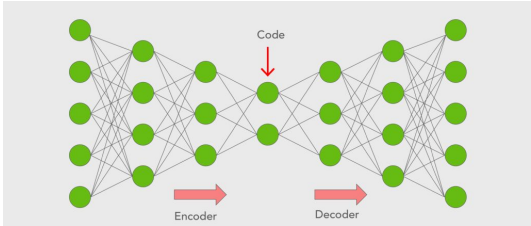
	Item 1	Item 2	Item 3
User 1	4	3	4
User 2	5	2	3
User 3	4	3	1

	Item 1	Item 2	Item 3
User 1		3	
User 2	5		3
User 3		3	



	Item 1	Item 2	Item 3
User 1	4	3	4
User 2	5	2	3
User 3	4	3	1

	Item 1	Item 2	Item 3
User 1		3	4
User 2			3
User 3	4	3	1



	Item 1	Item 2	Item 3
User 1	4	3	4
User 2	5	2	3
User 3	4	3	1

$U1 = [1, 3, 4, \dots X, , X] \rightarrow \text{AUTOENCODER} \rightarrow [1, 3, 4, \dots 2, 5, 1]$

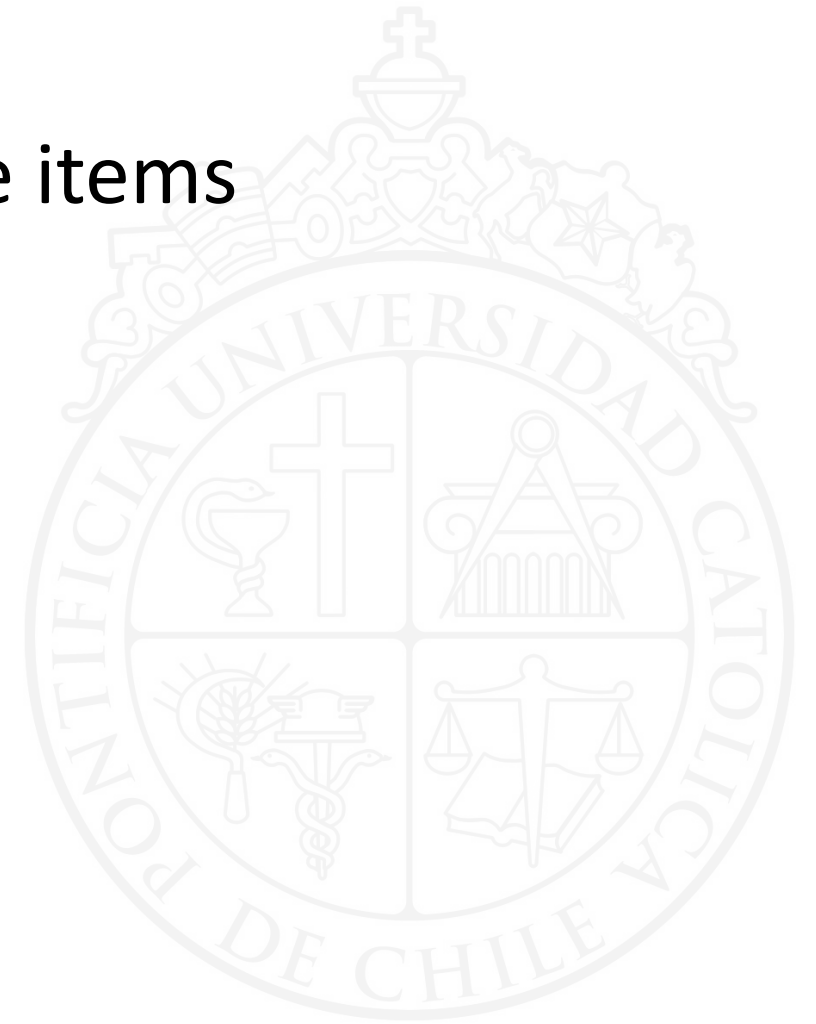
ITEM100 - 5

ITEM9 - 2

...

Resumen de la clase

1. Representación latente de usuarios e items
2. Reducción de dimensionalidad
 - Factorización matricial SVD
 - Factorización matricial FunkSVD
 - Autoencoders



Práctico de hoy

<https://drive.google.com/file/d/1Wm380dbgpS5HUUKRViHVvnNZqtNUTOcQ/view?usp=sharing>

